

```

accountsList[1].withdraw(345, 'MarysPassword') ❹
accountsList[1].deposit(100, 'MarysPassword') ❺

# Show the accounts
accountsList[0].show() ❸
accountsList[1].show()

# Create another account with information from the user
print()
userName = input('What is the name for the new user account? ')
userBalance = input('What is the starting balance for this account? ')
userBalance = int(userBalance)
userPassword = input('What is the password you want to use for this account? ')
oAccount = Account(userName, userBalance, userPassword)
accountsList.append(oAccount) # append to list of accounts

# Show the newly created user account
print('Created new account, account number is 2')
accountsList[2].show()

# Let's deposit 100 into the new account
accountsList[2].deposit(100, userPassword)
usersBalance = accountsList[2].getBalance(userPassword)
print()
print('After depositing 100, the user's balance is:', usersBalance)

# Show the new account
accountsList[2].show()

```

Listing 4-4: Modified test code to store objects in a list

We start by creating an empty list of accounts ❶. We create an account for Joe, store the returned value into the variable `oAccount`, and immediately append that object onto our list of accounts ❷. Since this is the first account in the list, Joe's account number is 0. Like at a real bank, any time that Joe wants to do any transactions with his account, he supplies his account number. We use his account number to show the balance of his account ❸, make a deposit ❹, then show the balance again ❺. We also create an account for Mary with account number 1 ❻ and perform some test operations on her account at ❼ and ❽.